UTN FRLP

Desarrollo de Software 2026

Control de Versiones y Trabajo Colaborativo + Html/CSS

Clase #02 - 6 de abril del 2026

Matías Corti - Martina García Améndola

Temas

1º Parte: Control de Versiones

- Comprender qué es el control de versiones y su importancia
- Conocer los fundamentos de Git
- Practicar con GitHub como plataforma colaborativa
- Incorporar herramientas para el desarrollo de software en equipo

2ª Parte: HTML5 + CSS3

- Introducción a HTML5 + CSS3
- Práctica con páginas estáticas

1º Parte
Control de Versiones

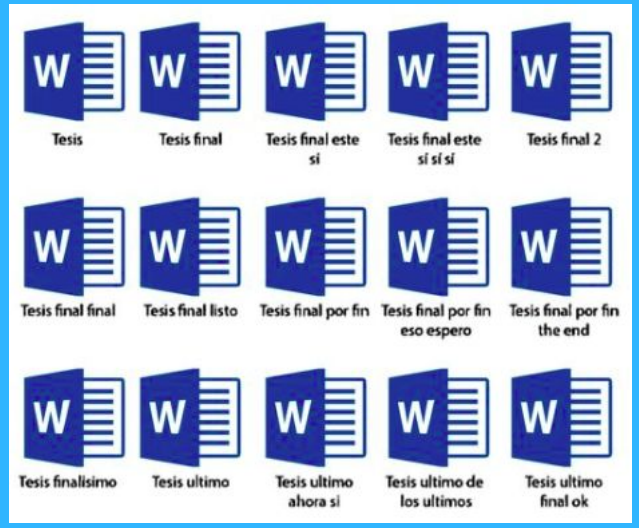
¿Qué es el control de versiones?

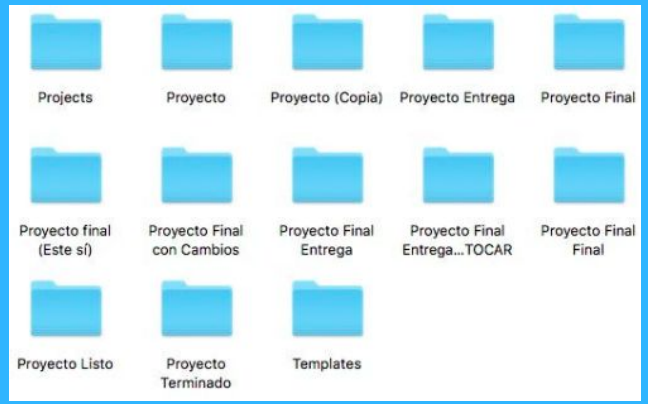
"Es como una máquina del tiempo para tu código"

- Sistema que registra cambios en archivos a lo largo del tiempo
- Permite volver a versiones anteriores
- Facilita el trabajo en equipo
- Registra quién realizó cada cambio y por qué

Antes del control de versiones...

- *Copias manuales con diferentes nombres*
- *Compartir archivos por email o USB*
- *Imposible saber quién cambió qué y cuándo*
- *Imposible trabajar en paralelo eficientemente*
- *Fusionar cambios era un proceso manual y propenso a errores*

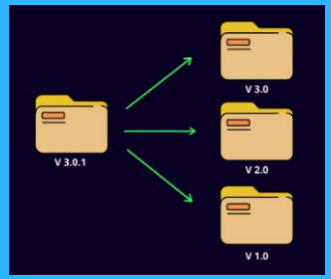




Tipos de sistemas de control de versiones

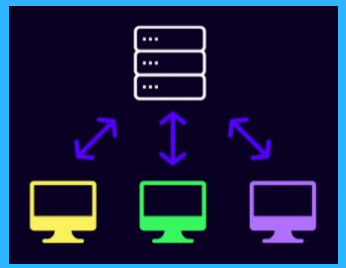
Sistemas locales

- Copias en diferentes directorios
- Bases de datos simples en la máquina local
- No permite colaboración entre varios desarrolladores



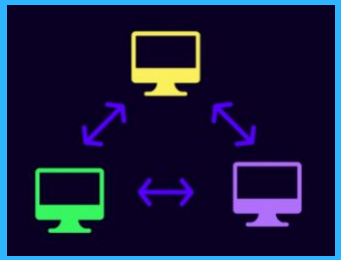
Sistemas centralizados (CVCS)

- Servidor central con todos los archivos versionados
- Ejemplos: CVS, Subversion (SVN)
- Problema: punto único de fallo



Sistemas distribuidos (DVCS)

- Copia completa del repositorio en cada cliente
- Ejemplos: Git, Mercurial
- Mayor robustez y flexibilidad



¿Qué es Git?
Historia y características



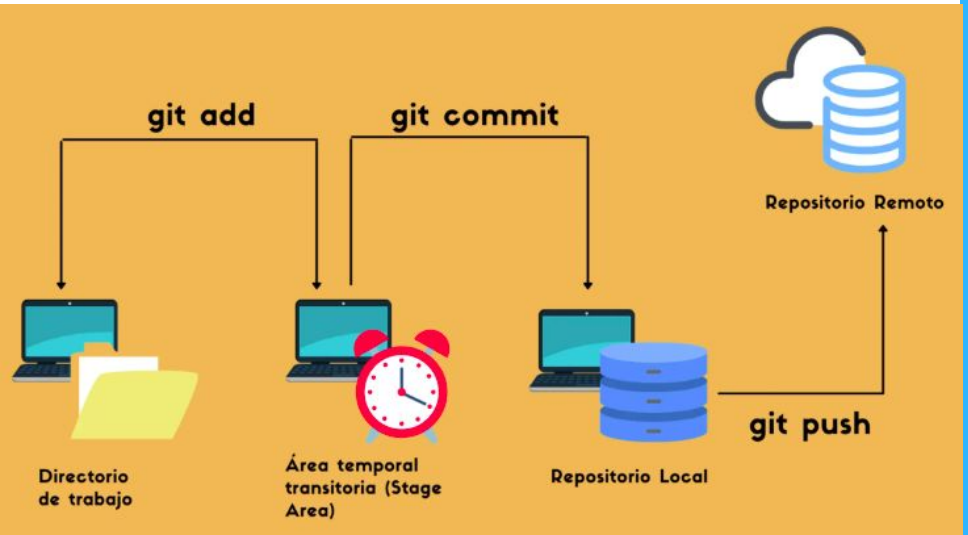
Creado por Linus Torvalds en 2005 para el desarrollo del kernel de Linux

Diseñado para ser:

- *Rápido*
- *Simple en diseño*
- *Descentralizado*
- *Capaz de manejar proyectos grandes*
- *Con fuerte soporte para desarrollo no lineal (miles de ramas paralelas)*

Google facebook Microsoft twitter android

Conceptos fundamentales de Git



Repositorio
Almacén de archivos y su historial de cambios

Working directory
Archivos con los que trabajamos actualmente

Staging area
Área intermedia donde preparamos los cambios

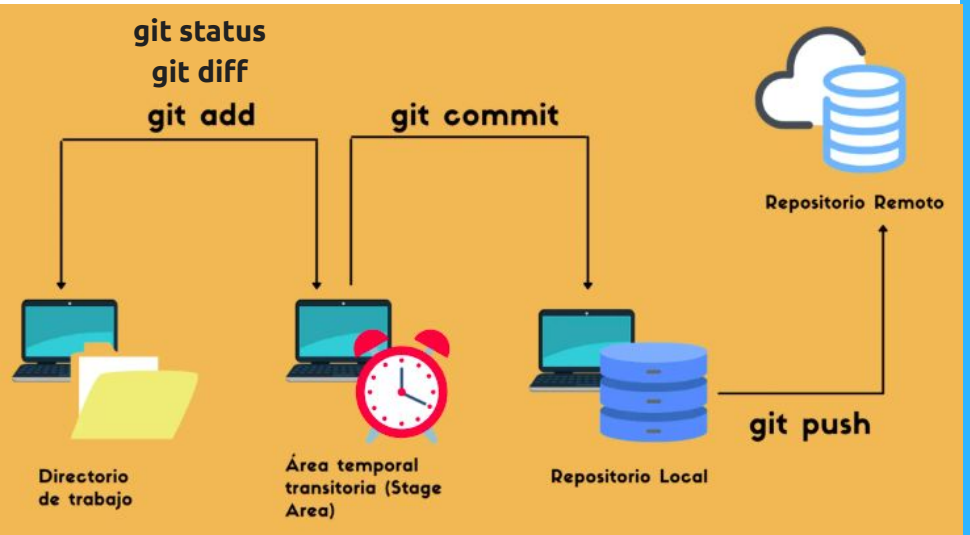
Commit
"Fotografía" del estado de los archivos en un momento dado

Branch
Línea independiente de desarrollo

Remote
Versión del repositorio alojada en un servidor(ej. GitHub)

Flujo de Trabajo Básico con Git

Descarga: https://git-scm.com/downloads



`git status`

Muestra el estado del directorio de trabajo y del área de staging

`git diff`

Permite ver cambios en archivos entre commits/staging area y el espacio de trabajo

`git add <archivo/s>`

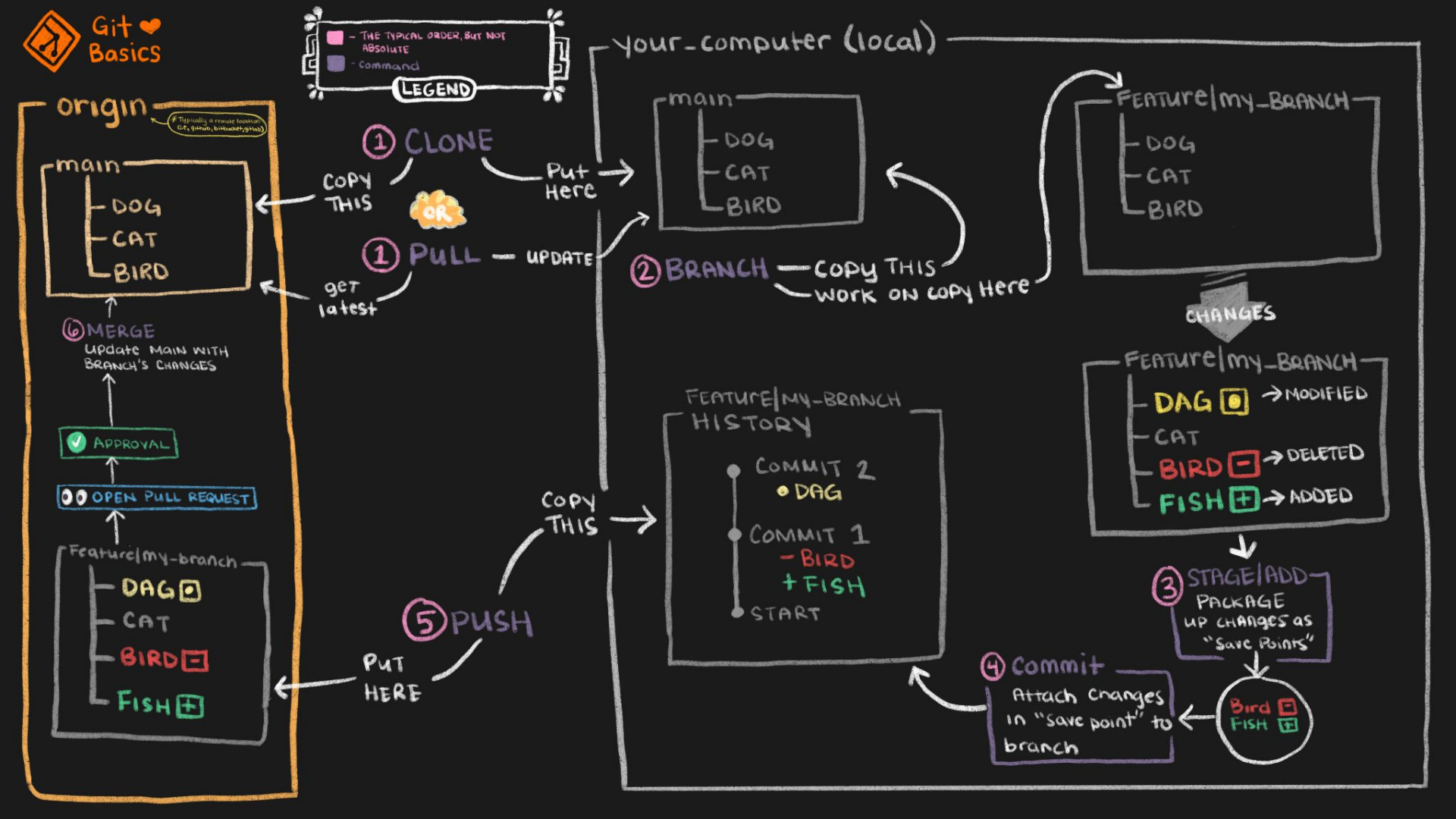
Añade uno o más archivos al área de staging

`git commit -m "mensaje"`

Guarda los cambios del área de staging en el historial del repositorio local

`git push`

Sube tus commits locales a la rama especificada del repositorio remoto


Git ♥ Basics
LEGEND
- THE TYPICAL ORDER, BUT NOT ABSOLUTE
- COMMAND
origin
* Typically a remote location (i.e. github, bitbucket, gitlab)
main
DOG
CAT
BIRD
1 CLONE
COPY THIS
PUT HERE
OR
1 PULL — UPDATE
GET LATEST
your-computer (local)
main
DOG
CAT
BIRD
2 BRANCH — COPY THIS
WORK ON COPY HERE
feature/my-branch
DOG
CAT
BIRD
CHANGES
feature/my-branch
DAG → MODIFIED
CAT
BIRD - → DELETED
FISH + → ADDED
3 STAGE/ADD
PACKAGE UP CHANGES AS "SAVE POINTS"
Bird -
Fish +
4 Commit
Attach changes in "save point" to branch
feature/my-branch HISTORY
COMMIT 2
• DAG
COMMIT 1
- BIRD
+ FISH
START
COPY THIS
5 PUSH
PUT HERE
feature/my-branch
DAG
CAT
BIRD -
FISH +
6 MERGE
Update main with branch's changes
APPROVAL
OPEN PULL REQUEST

Comandos básicos de Git

git init
Inicializa un nuevo repositorio en un directorio existente

git clone <url>
Copia un repositorio remoto a tu máquina local

git log
Muestra el historial de los últimos commits del branch

git branch
Lista las ramas locales

git checkout -b <nombre-rama>
Cambia a la rama especificada. También sirve para revertir los cambios de archivos en el directorio de trabajo *(sin: -b <nombre-rama>)*

git merge
Integra los cambios de la rama especificada en la rama actual

git fetch
Descarga las ramas y los commits del repositorio remoto especificado (sin mergearlas)

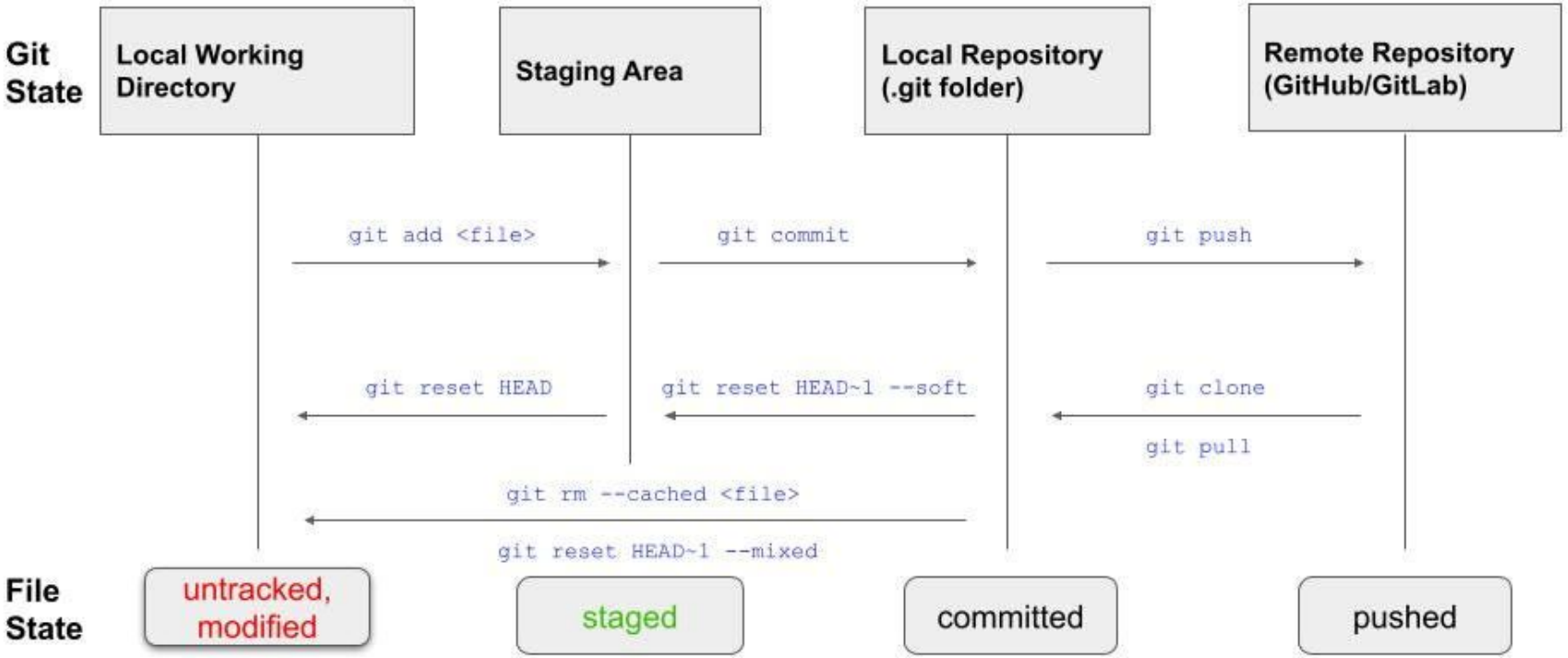
git pull
Descarga los cambios del repositorio remoto y los integra (merge) en tu rama local actual. Es la combinación de los dos anteriores.

Archivo importante: .gitignore

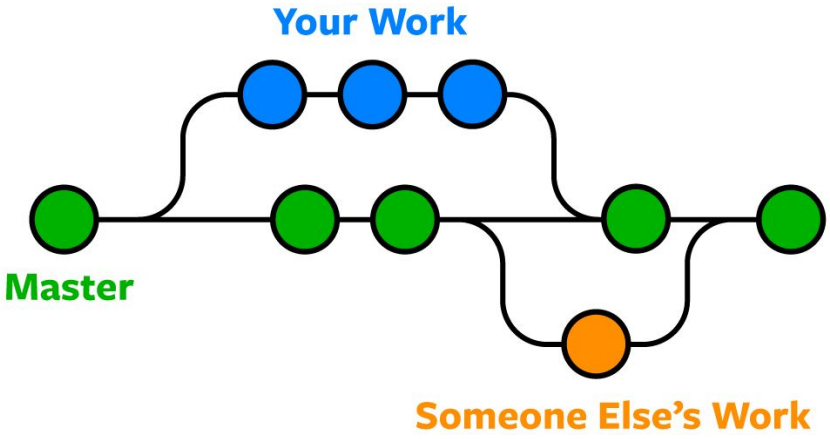
Le indica a Git qué archivos/carpetas ignorar

- Se pone en la raíz del repositorio
- Ignorar siempre:
  - Archivos generados automáticamente *(como logs, caché o temporales)*
  - `.env` *variables de entorno (datos sensibles)*
  - `node_modules/` o `vendors/` *Dependencias de proyectos*
- Ejemplo **.gitignore** básico:

```
node_modules/
.env
.DS_Store
*.log
```


Git State
Local Working Directory
Staging Area
Local Repository (.git folder)
Remote Repository (GitHub/GitLab)
git add <file>
git commit
git push
git reset HEAD
git reset HEAD~1 --soft
git clone
git pull
git rm --cached <file>
git reset HEAD~1 --mixed
File State
untracked, modified
staged
committed
pushed

Ramas en git
Branches



Permiten trabajar en paralelosin afectar la rama principal

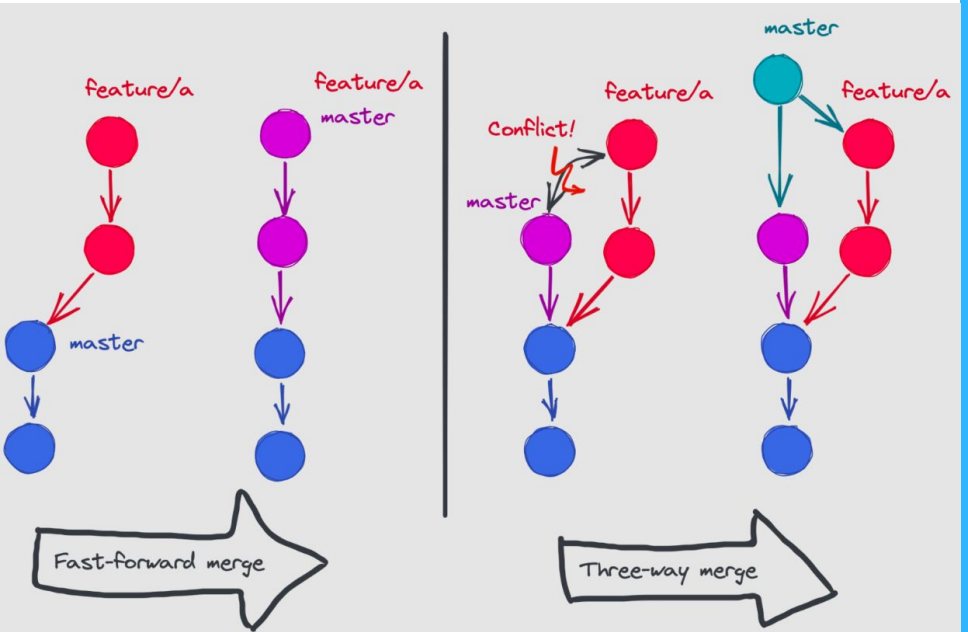
Ideal para:

- Desarrollar nuevas funcionalidades
- Corregir errores (bugfixing)
- Experimentar sin afectar el código principal

```
# Crear una nueva rama
git branch nombre-nuestra-rama
# Cambiar a esa rama
git checkout nombre-nuestra-rama

# Lo mismo en una sola operación
git checkout -b nombre-nuestra-rama
```

Fusión de ramas (Merge)



Permite integrar ramas entre sí

Existen dos tipos principales de merge:

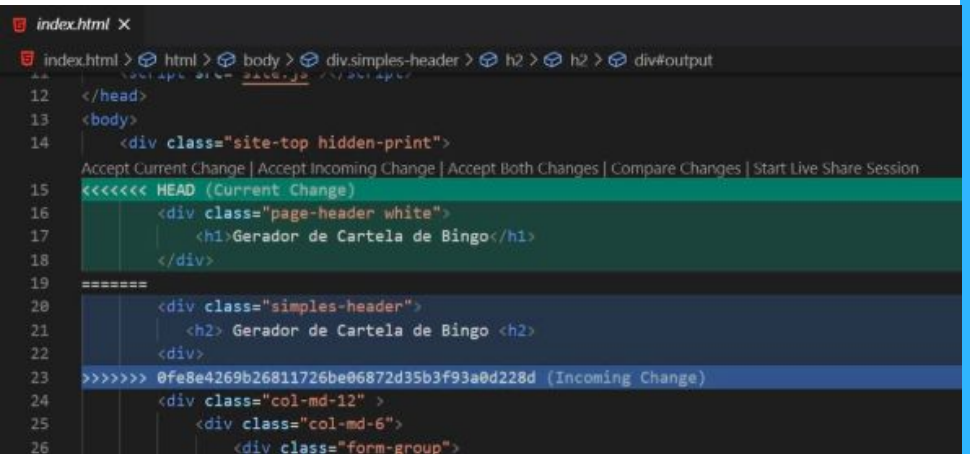
- **Fast-forward merge:** Cuando no hay cambios divergentes. Simplemente "adelanta" la rama destino hasta nuestra rama

- **Three-way merge:** Cuando ambas ramas tienen cambios diferentes. Git crea un nuevo commit que combina los cambios de ambas ramas

```
# Pasamos a la rama "destino"
git checkout master

# Cambiar a esa rama
git merge nombre-nuestra-rama
```

Conflictos en fusión de ramas



Ocurren cuando:

- Se modifican las mismas líneas en diferentes ramas
- Git no puede determinar automáticamente qué cambio conservar

```
<<<<<<< HEAD
Cambios en la rama actual
=======
Cambios en la rama que estamos fusionando
>>>>>>> rama-a-fusionar
```

Resolución manual:

1. Editar archivos para arreglar conflictos
2. Marcar como resueltos (git add)
3. Completar el merge (git commit)

GitHub: Plataforma colaborativa



*Es una **plataforma web para alojar repositorios Git**, que añade funcionalidades para facilitar la colaboración.*

Issues: permiten hacer seguimiento de tareas, bugs o mejoras

Pull Requests: son una forma estructurada de proponer cambios y revisarlos en equipo antes de integrarlos

Actions: permite automatizar flujos de trabajo como pruebas o deploys

Projects: facilita la gestión visual de tareas con tableros tipo Kanban

Pages: permite publicar sitios web directamente desde un repositorio

Recursos Adicionales



Documentación oficial de Git

Descargas oficial de Git

Pro Git Book Libro completo y gratuito sobre Git

Try Git Tutorial interactivo para aprender Git

Oh my git! Un juego open source para aprender Git

Resumen



✅ **Control de versiones**
Historial completo + Colaboración

✅ **Git**
Distribuido, rápido, estándar de la industria

✅ **Flujo diario**
modificaciones → *add* → *commit* → *push*

✅ **GitHub**
Es donde vive el código en la nube
(Existen otras plataformas similares)

⏭️ **Más adelante veremos**
branches, Pull Requests, trabajo en equipo

2ª Parte

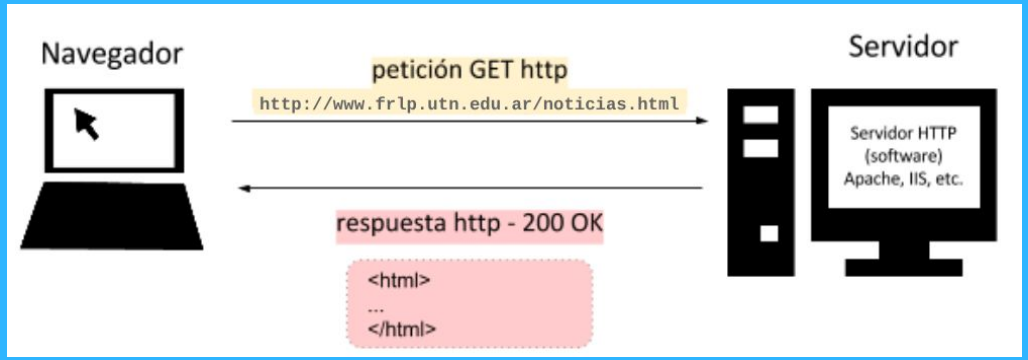
HTML5 + CSS3

¿Cómo funciona la web?

¿CÓMO FUNCIONA LA WEB?



- El ***Cliente*** (por ejemplo un *Navegador web*) solicita un *documento* (archivo HTML, CSS, etc) al ***Servidor*** mediante el protocolo HTTP
- El ***Servidor***, en caso de ser posible, se lo envía como respuesta
- El ***Cliente***, al recibirlo, lo *parsea* (interpreta) de arriba hacia abajo y de izquierda a derecha, *renderizando* (mostrando) en pantalla los Elementos que contiene el documento



Apunte de HTML y CSS: Cómo Funciona



HTML



HTML + CSS



HTML + CSS + JavaScript



HTML5 Estructura Semántica

Estructura ≠ Presentación

Accesibilidad: Lectores de pantalla

SEO: Mejor indexación

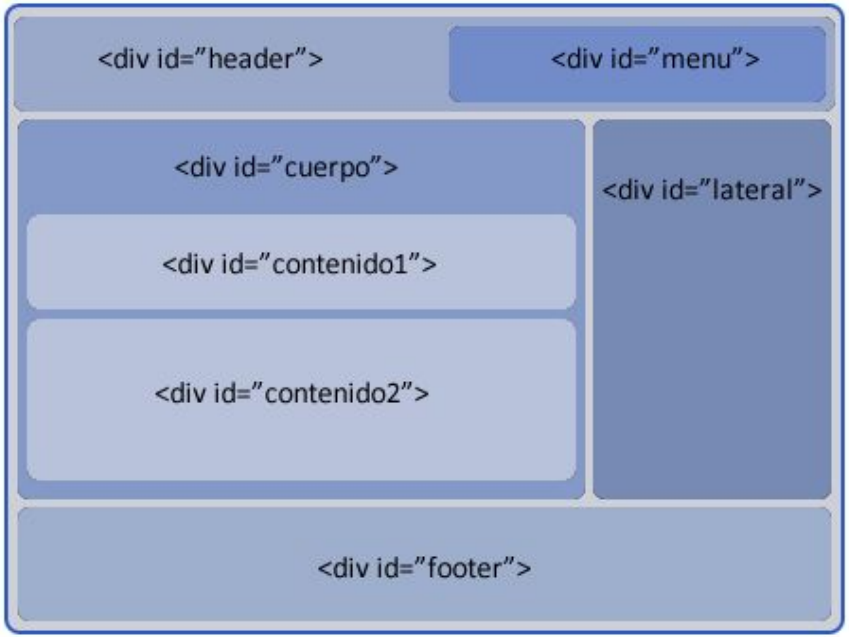
Mantenimiento: Código más claro

Etiquetas Semánticas

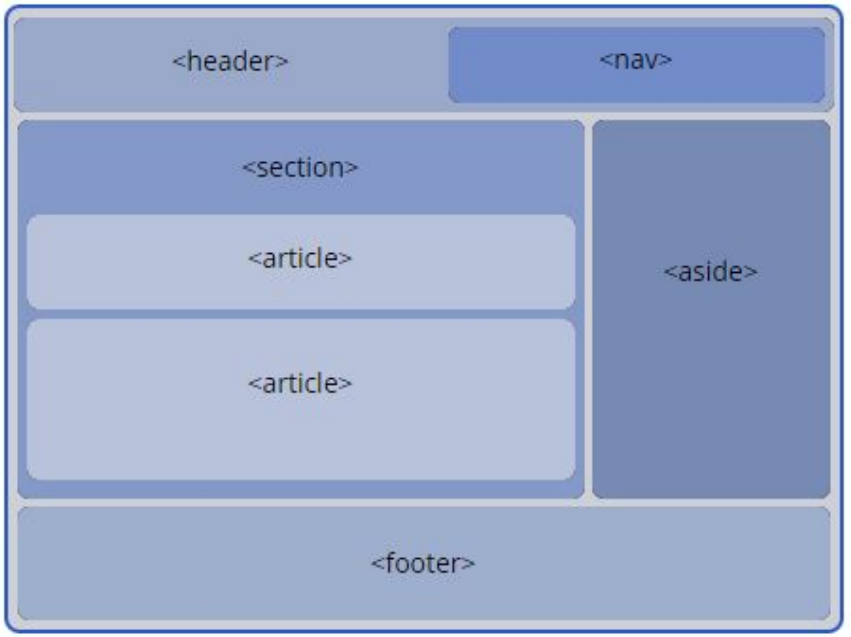
```
<header>    Cabecera de página/sección
<nav>       Navegación principal
<main>      Contenido principal
<section>   Sección temática
<article>   Contenido independiente
<aside>     Contenido relacionado
<footer>    Pie de página/sección
```

Apunte de HTML y CSS: Estructura

HTML4



HTML5



HTML 5 - Estructura Básica

```
<!DOCTYPE html>     < indica que es HTML5
<html lang="es">    < "lang" define que el lenguaje de la página es español

<head>    < metadata interna (no visible)
   <meta charset="UTF-8">       < codificación para acentos y ñ
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Mi Página</title>     < título (se ve en la pestaña)        ^ para que se vea bien en celulares
   <link rel="stylesheet" href="styles.css">        < link a la hoja de estilos CSS
</head>

<body>    < información visible de la web
   <header>
       <h1>Título Principal</h1>
       <nav><!-- Navegación --></nav>
   </header>
   <main>
       <section><!-- Contenido principal --></section>
   </main>
   <footer><!-- Pie de página --></footer>
</body>

</html>
```

Apunte de HTML y CSS: Estructura



Etiquetas HTML más usadas

Etiqueta	Para qué
`<h1>` a `<h6>`	Títulos (`<h1>` sólo uno por página)
`<p>`	Párrafos
`<a href="...">`	Links/enlaces
`<img src="..." alt="...">`	Imágenes
`<ul>`, `<ol>`, `<li>`	Listas (sin orden / con orden)
`<div>`	Contenedor genérico (bloque)
`<span>`	Contenedor genérico (en línea)
`<strong>`, `<em>`	Negrita semántica / Itálica semántica
` `	Salto de línea
`<table>`, `<tr>`, `<td>`	Tablas

Apunte de HTML y CSS: Etiquetas



HTML5

Formularios

Formularios = interacción con el usuario

Apunte de HTML y CSS: Formularios

Son los elementos HTML que permiten al usuario ingresar y enviar datos

Ejemplo:

```
<form>
    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre"
           placeholder="Tu nombre" required>

    <label for="email">Email:</label>
    <input type="email" id="email"
           placeholder="email@ejemplo.com" required>

    <label for="mensaje">Mensaje:</label>
    <textarea id="mensaje" rows="3"></textarea>

    <button type="submit">Enviar</button>
</form>
```



HTML5

Formularios

Formularios = interacción con el usuario

Apunte de HTML y CSS: Formularios

Ejemplos de inputs:

```
<!-- Campo de texto obligatorio con placeholder -->
<input type="text" placeholder="Ingrese su nombre" required>

<!-- Campo de correo electrónico con validación de formato -->
<input type="email" placeholder="usuario@ejemplo.com"
required>

<!-- Campo de contraseña con longitud mínima -->
<input type="password" placeholder="Mínimo 6 caracteres"
minlength="6" required>

<!-- Campo numérico con valores mínimo y máximo -->
<input type="number" placeholder="Edad (18-99)" min="18"
max="99" required>

<!-- Campo de fecha con rango de fechas permitido -->
<input type="date" min="2000-01-01" max="2025-12-31" required>

<!-- Campo de archivo que solo acepta imágenes -->
<input type="file" accept="image/*" required>

<!-- Botón para enviar formulario -->
<button type="submit">Enviar</button>
```

Placeholder:

Texto de ejemplo que aparece dentro de un campo vacío



CSS3 Diseño Moderno

De esto...

Página de 1990

...a esto

Aplicación Moderna 2025

Diseño limpio, responsive, interactivo

Conceptos Importantes

Box Model:
margin, border, padding, content

Flexbox:
Layout flexible y responsive

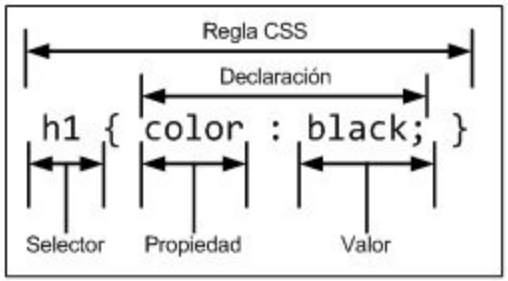
CSS Grid:
Layouts complejos en 2D

Variables CSS:
Reutilización y mantenimiento

Media Queries:
Responsive design

CSS3

Introducción



Apunte de HTML y CSS: Intro CSS

Cómo incluir CSS en la página:

- Archivo externo **.css**
- Vincular a la página con la etiqueta **<link>** en el **<head>**
- Se puede usar en línea o con la etiqueta **<style>** (pero no es lo recomendado)

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mi Página</title>
    <link rel="stylesheet" href="styles.css">
</head>
```

Selectores

Cómo indicar a CSS a qué elementos aplicar los estilos

```
p { <declaración> }        → por etiqueta, el ejemplo: a todos los <p>
.clase { <declaración> }   → por clase (el más usado)
#id { <declaración> }      → por id (único en la página)
```

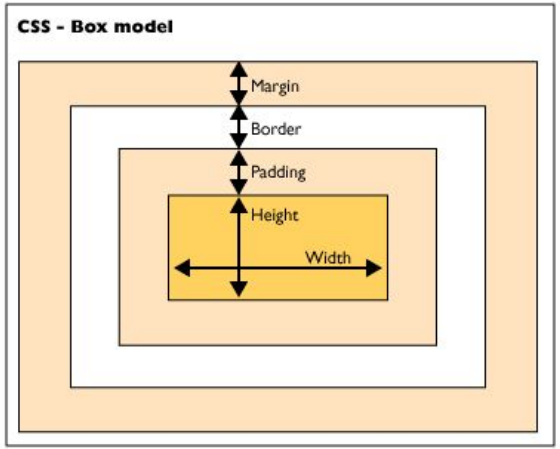


CSS3 Propiedades básicas

```css
color: #333;                    /* color del texto */
font-family: Arial, sans-serif; /* tipografía */
font-size: 1rem;                /* tamaño */
text-align: center;             /* alineación */
background-color: #f5f5f5;      /* fondo */
padding: 1rem;                  /* espacio interno */
margin: 1rem;                   /* espacio externo */
border: 1px solid #ccc;         /* borde */
border-radius: 8px;             /* esquinas redondeadas */
```

Apunte de HTML y CSS: Propiedades CSS

CSS3
Box Model



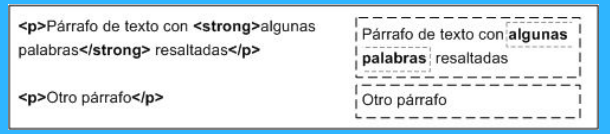
[Apunte de HTML y CSS: Box Model]

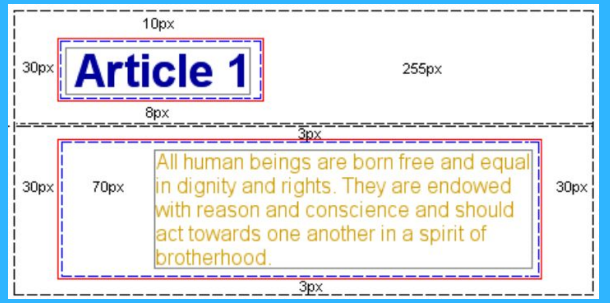
Es el comportamiento de CSS que hace que todos los elementos se representen mediante cajas rectangulares.

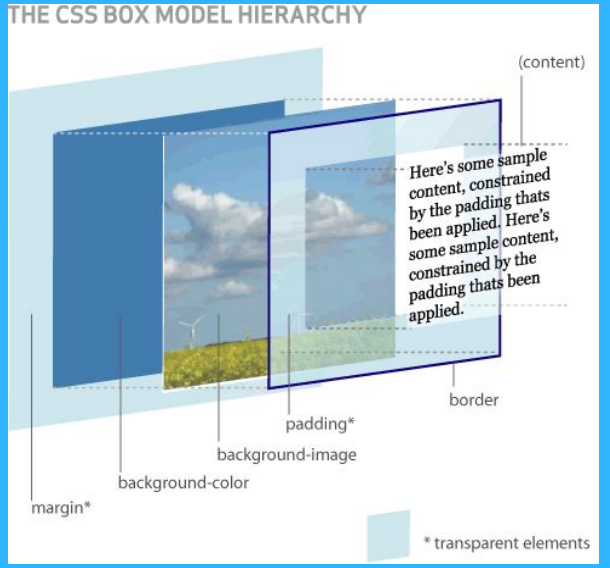
Los navegadores crean y colocan las cajas de forma automática, CSS permite modificar todas sus características.

Cada caja está formada por cuatro partes y su orden de visualización desde el punto de vista del usuario son las siguientes:

1. **Contenido**: El contenido HTML del elemento
2. **Relleno**: Espacio libre opcional entre el contenido y el borde.
3. **Borde**: Línea que encierra completamente el contenido y su relleno.
4. **Margen**: Separación existente entre la caja y el resto de cajas adyacentes.









CSS3 Variables (Custom Properties)

✅ Fácil mantenimiento
✅ Consistencia
✅ Temas dinámicos

Apunte de HTML y CSS: Variables CSS

Definición

```
:root {
  --primary-color: #3498db;
  --secondary-color: #2c3e50;
  --border-radius: 8px;
  --shadow: 0 2px 10px rgba(0, 0, 0,
0.1);
}
```

Uso

```
button {
  background: var(--primary-color);
  border-radius: var(--border-radius);
  box-shadow: var(--shadow);
}

.bkg-secondary {
  background: var(--secondary-color);
}
```



CSS3 Flexbox

Adiós floats y positioning!

[Apunte de HTML y CSS: Flexbox]
[MDN Flexbox]
[Juego: Fexbox Froggy]

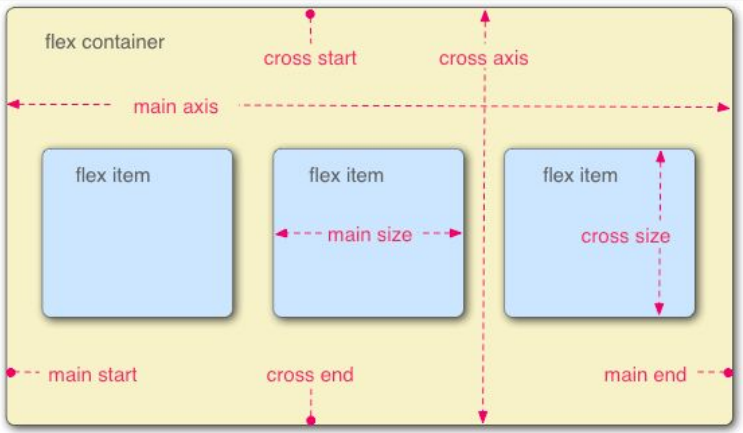
Ventajas

Alineación fácil: vertical y horizontal

Distribución: espacio automático

Responsive: se adapta automáticamente

Orden flexible: sin modificar HTML




```css
.container {
   display: flex;
   flex-direction: row;
   /* column */
   justify-content: center;
   align-items: center;
   gap: 1rem;
}

.item {
   flex: 1;
   /* Crece proporcionalmente */
}
```



CSS3 Diseño Responsive

Un sólo código base

⬇

Múltiples dispositivos

[Apunte de HTML y CSS: Responsive](#)
[MDN Media Queries](#)

```css
/* Mobile First Approach */
.container {
    padding: 1rem;
    flex-direction: column;
}

/* Tablet */
@media (min-width: 768px) {
    .container {
        padding: 2rem;
        flex-direction: row;
    }
}

/* Desktop */
@media (min-width: 1024px) {
    .container {
        max-width: 1200px;
        margin: 0 auto;
    }
}
```

Resumen HTML + CSS



✅ **HTML5**
Estructura semántica (header, main, footer, section)

✅ **CSS3**
Usar archivo externo, selectores por clase

✅ **Box Model**
content → padding → border → margin

✅ **Flexbox para layouts, variables CSS para consistencia**

✅ **Responsive con media queries (mobile first)**

📖 **Referencia completa:** Apuntes HTML y CSS



Práctica

HTML5 + CSS3 + GIT



Práctica

HTML5 + CSS3 + Git

- 6 ejercicios (0 a 5) con dificultad creciente
- Cada ejercicio **en su carpeta** dentro del repositorio:

```
ds2026-actividades/
├── README.md
├── .gitignore
└── ejercicios/
    └── c02-html-css/
        ├── ej1-primera-pagina/
        ├── ej2-cv-multipagina/
        ├── ej3-css-externo/
        ├── ej4-flexbox-variables/
        └── ej5-responsive-form/
```

- Cada ejercicio terminado = **un commit** (con mensaje descriptivo)
- ⏱️ **En clase:** arrancar desde el Ejercicio 0, llegar hasta donde puedan
- 📅 **Durante la semana:** completar hasta el Ejercicio 4 como mínimo
- ⭐ Ejercicio 5: bonus para los que quieran ir más allá
- 📖 Material de consulta: Apuntes HTML y CSS + MDN Web Docs

Ejercicios Prácticos

Ejercicio 0 — Setup Git 🔧

- Crear cuenta personal en GitHub
- Crear repositorio `ds2026-actividades` (público)
- Clonar localmente → crear README.md (nombre + comisión) → crear .gitignore
- Primer commit + push → verificar en GitHub
- Commit: `"Ejercicio 0: setup del repositorio"`

Ejercicio 1 — Primera página HTML 📄

- Crear `ej1-primera-pagina/index.html`
- Estructura HTML5 completa (DOCTYPE, head, body)
- Un `<h1>`, un párrafo, una imagen, una lista, un link externo
- Commit: `"Ejercicio 1: primera página HTML"`

Ejercicio 2 — CV multipágina 📑

- Crear 4 páginas: index, estudios, experiencia, hobbies
- Navegación entre todas (links internos)
- Usar: h1-h3, p, ul, ol, img, a, strong, em
- Convenciones: minúsculas, indentación, comillas dobles
- Commit: `"Ejercicio 2: CV multipágina con navegación"`

Ejercicio 3 — CSS externo en CV 🎨

- Copiar el **CV del ejercicio 2** a `ej3-css-externo/`
- Crear `styles.css` y vincularlo con `<link>`
- Colores, tipografías, márgenes, paddings
- Al menos 1 selector de clase y 1 de etiqueta
- Commit: `"Ejercicio 3: CSS externo"`

Ejercicio 4 — CV con Flexbox y variables CSS 📐

- Copiar el **CV del ejercicio 3** a `ej4-flexbox-variables/`
- Definir variables CSS en `:root` (mínimo 3)
- Reorganizar el layout con Flexbox (ej: navbar horizontal, secciones en grilla)
- Aplicar `box-sizing: border-box` global
- Bordes redondeados, sombras usando las variables
- Commit: `"Ejercicio 4: flexbox y variables CSS"`

Ejercicio 5 — Responsive + Formulario ⭐

- Copiar el **CV del ejercicio 4** a `ej5-responsive-form/`
- Agregar `contacto.html` con formulario
- Validaciones HTML5 (required, type="email", minlength)
- Media queries (layout adaptable a mobile)
- Commit: `"Ejercicio 5: formulario y responsive"`



Recursos y Referencias

📖 Documentación

- Documentación oficial de Git
- Pro Git Book
- MDN Web Docs
- CSS-Tricks

🛠️ Herramientas

- Descargas oficial de Git
- VS Code
- Chrome/Firefox/Edge "Dev Tools" (F12)

🎮 Práctica

- Try Git
- Oh my git!
- CodePen
- Flexbox Froggy
- Grid Garden

Actividades de la semana

Actividad individual de la semana

1º instancia hasta el **12/04/2026**
2º instancia: ***10/05/2026*** *(Cierre Bloque 1)*

[AI-C02 - Git + HTML/CSS]
Completar Ejercicios Prácticos

- Completar al menos los **ejercicios del 0 al 4**
- Si llegan hasta el ejercicio 5 suman más puntos
- Cada ejercicio en su carpeta, cada uno con su commit
- Subir todo al repo ds2026-actividades en GitHub
- Verifiquen que el **repositorio sea público** para que los podamos acceder los docentes

Una vez completada la entrega, escribir en Discord un mensaje en el canal **#actividades**, dentro del Hilo: ***"AI-C02 - Git + HTML/CSS"***, *que incluya la URL propia del repositorio individual en GitHub.*

Actividad grupal de la semana

Hasta el **12/04/2026**
(inclusive)

[AG-C02 - Cliente TP Grupal]

Definir y comunicar el cliente del grupo para el Trabajo Práctico

- Definir el **cliente real** para el TP grupal
- Enviar en Discord un **mensaje único por equipo,** dentro del canal **#actividades**, en el Hilo: ***"AG-C02 - Cliente TP Grupal"***, indicando:
  - Número de Grupo asignado
  - Quién es el cliente
  - Tema/dominio del proyecto
  - Breve descripción (2-3 líneas)

Recuerden: *El cliente debe ser un establecimiento real al que puedan ofrecerle una solución concreta.*